

Bath Planner

Prompts & Workflow Documentation

AI-Assisted Development Log — KOHLER AI Bathroom Designer & Planner, Case-Study Submission

Live app: kohlerbathplanner.vercel.app • Repository: github.com/Alok49225/kohlerbathplanner • AI system used throughout: Claude (Anthropic), via Claude Code

1. Overview

Bath Planner was built end to end in an extended conversational collaboration with Claude, using Claude Code as the development environment. This document details the prompts, standing instructions, and workflows that shaped that collaboration — not a transcript dump, but the actual methodology, illustrated with real, verbatim prompts drawn directly from the build. No custom system prompt was engineered for this project beyond a small set of standing rules the developer established once and Claude followed for the rest of the build (Section 2) — these rules functioned as persistent, de facto instructions across every task.

Where each grading criterion is addressed in this document:

Criterion	Covered in
Approach & Innovation (45%)	Sections 2, 3, 4 — the workflow itself, the build history, and the deliberate no-live-LLM design decision
Technical Execution (25%)	Sections 2 & 3 — spec-first, test-backed, root-cause-driven engineering across the full build
UX & Feasibility (20%)	Sections 3.8 & 3.11 — a user-authored UX redesign, and real bug reports from actually using the shipped app, each diagnosed and fixed
Business & Sustainability (10%)	Section 5 — water-efficiency scoring in the solver and bundle summary

2. Development Methodology & Workflow Patterns

The same handful of practices were applied consistently across all 35 build-board tasks, every user-reported bug, and deployment — not ad hoc per task. These functioned as the project's real “system instructions”: rules established once in conversation and applied by Claude for the rest of the build.

Dependency-first task ranking

Before building anything, the 35-task build board was reordered by actual dependency and implementation ease — schemas before the code that consumes them, the core solver before any UI that calls it, independent pieces first — rather than following the board's original listing order.

Spec-before-code gate

For every task, Claude presented a concrete spec (what would be built, where, and why) and waited for explicit approval before writing any code. No implementation began on an unapproved plan, across 35 tasks and every subsequent bug fix — the founding instruction is quoted in full in Section 3.4.

Build modular, integrate later

Each task was built and tested as a standalone, controlled unit with no wiring into other layers until a dedicated integration pass. This kept every component independently testable, at the cost of the running app looking unfinished for most of the build — page composition was deliberately its own, later task (Section 3.10; the founding instruction is quoted in Section 3.4).

Reproduce before fixing — never guess

Every bug report, whether from testing or from the developer using the live app, was reproduced first and root-caused by direct inspection (live DOM measurement, code tracing, or a minimal repro) before any fix was proposed. Section 3.11 shows this pattern applied to seven real, distinct bugs.

Blast-radius check before touching finished work

Before modifying any already-completed module, its consumers were located (by grep and by reviewing existing tests) so a fix's full impact was known before the fix was written, not discovered after.

Live verification, not just green tests

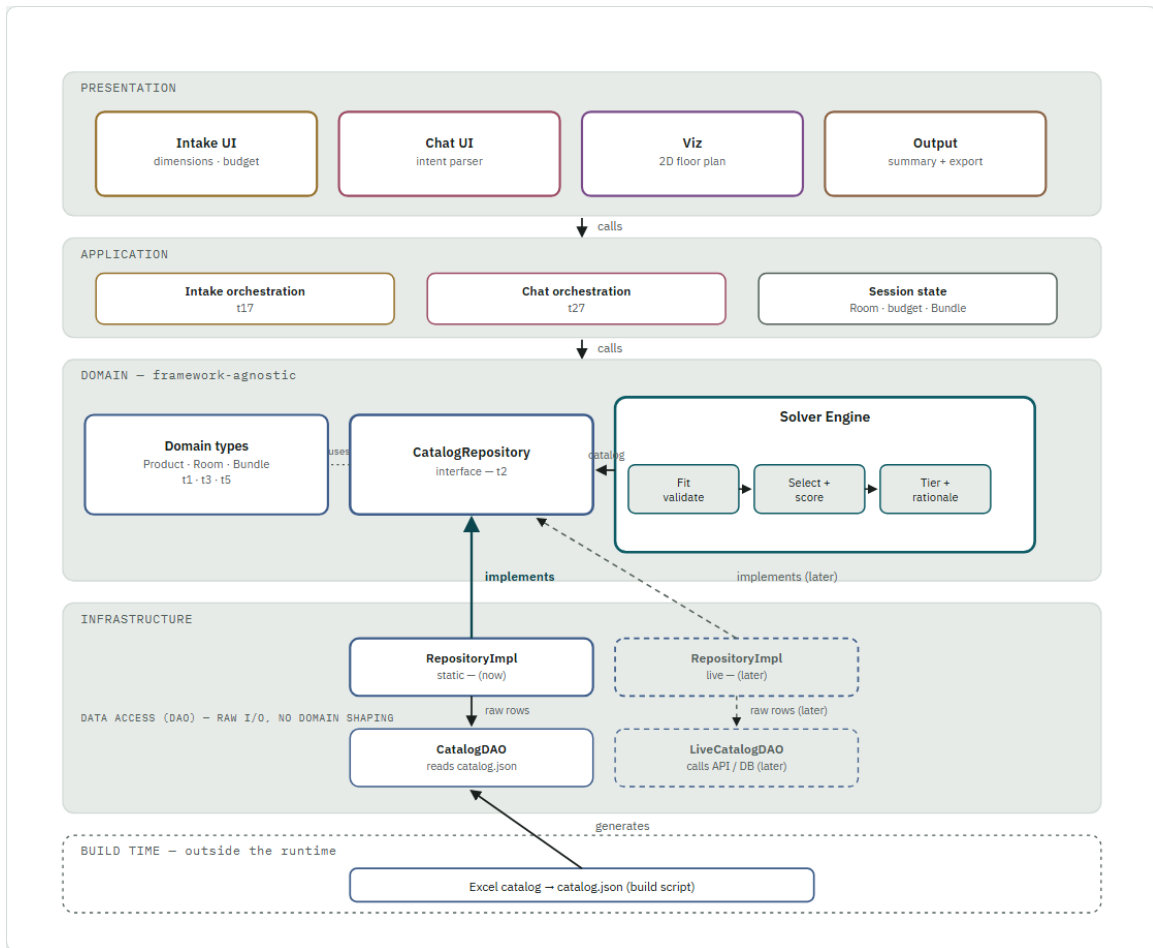
UI-affecting changes were verified by actually driving the running app in a browser — clicking, measuring real coordinates, reading real console output — in addition to the automated test suite, both locally and, for the final changes, on the live production deployment.

Explicit approval before every commit

Nothing was committed to git without the developer explicitly asking for it, task by task, for the entire build.

The resulting architecture

These rules produced a real layered architecture, not just a described one: Presentation calls Application calls Domain, and Infrastructure implements an interface Domain owns rather than being called by it directly — the one deliberate seam that lets the static demo catalog be swapped for a live inventory feed later without touching the solver or the UI. Published in full, with an accompanying data-flow diagram, as *architecture-diagram.html* in the repository — the actual exchange that produced it is quoted in Section 3.6.



Every arrow points down except one: *RepositoryImpl* implements *CatalogRepository*, the accent-colored inversion that makes the static-to-live swap a one-layer change instead of a rewrite.

3. The Build, in Prompts

The sections below follow the actual chronology of the project. Each includes a real, verbatim prompt from the build and the outcome it produced.

3.1 The Founding Prompt

DEVELOPER

"I am participating in a competition where i have to solve a case study. This case is marked highest for innovation and approach and second for technical and business approach.

Case study - KOHLER AI Bathroom Designer & Planner

Objective: Build an interactive AI design assistant that takes a customer's constraints and automates personalized product bundle recommendations. Key Inputs: Bathroom dimensions (ft x ft / layout or image), budget limits, aesthetic themes (e.g., Minimalist Modern, Classic Luxury, Japanese Zen), and device catalog specifications. Expected Outcome: An intelligent recommendation engine that outputs optimized product combinations (faucets, smart toilets, thermostatic showers, vanities) fitting exact physical space and budget parameters. If possible a 2D or 3D representation of the bathroom layout with the selected products.

give me ideas how to solve this problem"

Outcome: the origin of the entire project. Every artifact, task, and line of code in this document is downstream of this single message — including the grading-weight framing (innovation & approach first, technical & business second) that later shaped which features got built first and how this very document is organized.

3.2 Acting as Product Owner — the Blueprint

DEVELOPER

"act as a project owner or product manager and define all the features required and then we can solve the technical problem later on"

Outcome: produced the Blueprint artifact — an 8-section product requirements outline (vision & problem, 3 personas, a 6-step user journey, a MoSCoW-prioritized feature catalog across 6 modules, non-functional requirements, success metrics, a 3-phase roadmap, and explicit out-of-scope items) — deliberately separating what to build from how, exactly as asked.

3.3 Breaking Down the MVP — the Build Board

DEVELOPER

"we will be now be developing phase 1 MVP . divide the development of MVP in smaller tasks"

Outcome: produced the Build Board artifact — all 35 Phase-1 tasks broken out across 6 workstreams (data, engine, intake, viz, chat, output), later reordered into the dependency-first build sequence quoted in Section 3.5.

3.4 The Governing Rules

Two instructions given right as implementation began ended up governing literally everything that followed.

DEVELOPER

"from now on you won't start coding before my approval , after specs we will review it and discuss it . also write in a structure so that all the task can be connected to make a proper model"

Outcome: established the spec-before-code gate applied to all 35 tasks and every bug fix from this point on (Section 2), plus an explicit structural requirement — that separately-built tasks still had to connect into one coherent model — which is the seed of the later integration task (Section 3.10).

DEVELOPER

"each task should be modular and we will integrate after all the task will be completed"

Outcome: the exact origin of the build-modular-integrate-later rule (Section 2): it's why the visualization, output, and most of the conversational-layer tasks (t20 through t30) were each built and tested as standalone, unwired components until one dedicated, later integration pass composed them into the real running app (Section 3.10).

3.5 Planning & Task Ranking

DEVELOPER

"once again we will build one task at a time, first rank which task to do according to dependency and ease of coding"

Outcome: the 35-task build board (6 workstreams: data, engine, intake, viz, chat, output) was reordered into a real dependency-first build sequence (t1→t3→t5→t2→t4...→t35), used for the rest of the project.

3.6 A Real Architecture Decision — Data Layer Separation

A short back-and-forth, not a single instruction, is the real origin of the layered architecture shown in Section 2.

DEVELOPER

"from engineering and software development aspects ,is it a not a better practice to have a separate data layer or database"

Outcome: raised the architecture question directly, rather than leaving it to be inferred from a generic best-practice default.

DEVELOPER

"though we dont need a full fledged db but from architecture point of view is it not better to make separation now"

Outcome: narrowed the answer to exactly what was actually built: an interface seam (*CatalogRepository*) without a real database — the deliberate scope-matched middle ground now documented as the one seam in the architecture diagram (Section 2), designed for a future live-inventory swap.

3.7 Solver Engine (t4–t13)

DEVELOPER

"give the specs for #1"

Outcome: each engine task followed the same loop: Claude proposed a spec, the developer replied "keep going with the ranked list" or asked follow-up questions, then Claude built and tested it. This produced the compatibility-rules engine, theme-fit scorer, clearance/fit validator, budget selector, multi-objective bundle scorer, tier generator, and pin-and-resolve — the full solver, 123/123 tests passing by the end of the workstream.

3.8 User-Authored UX — the Legend & Hover Redesign

Not every prompt in this document is a bug report or an approval — this one is a direct UX spec, authored by the developer, not proposed by Claude.

DEVELOPER

"will it not be better to show every thing in legend and when cursor hover over a particular component/ fixture and a white box appears that contains the name of the said component at z axis +1, and have a legend seperately for all teh components, without any text on the actual map for now"

Outcome: replaced an earlier inline-label design (text drawn directly on the floor plan, which collided and overflowed at the room's edges) with exactly the interaction now shipped: a hover tooltip per fixture plus one separate, always-visible legend below the plan — the actual FixtureLayer design in production.

3.9 Verifying Change Impact Before Being Asked To

DEVELOPER

"will this we affected because of new features added"

Outcome: Claude traced the actual call graph rather than assuming: confirmed the chat intent parser and orchestration were untouched by a recent plumbing-point change, and explained why (different, non-overlapping files). Blast-radius verification was applied even when only asked as a question, not requested as a task.

3.10 Integration — Composing the Built-but-Unwired App

DEVELOPER

"will it can be considered as integration part"

Outcome: confirmed that composing the floor plan, fixture layer, tier switcher, bundle summary, and chat panel into one real page — every piece built standalone since t19–t30 but never wired together — was exactly the deferred integration task, and prioritized it immediately: it was the single biggest gap between what was built and what an evaluator would actually see running.

3.11 Real Bugs, Found by Actually Using the App

Once the app was integrated, the developer used it directly and reported issues in plain language, exactly as a real user would — not as ticket-formatted bug reports. Each one was reproduced and root-caused before any fix was proposed.

DEVELOPER

"bundle cost shows \$infinity"

Outcome: traced to Infinity being reused as a generic sentinel for three structurally different infeasibility reasons (no eligible products, a missing plumbing point, over budget), which had collapsed all three into one meaningless number. Replaced with a discriminated union threaded through the solver, the orchestration hook, and three distinct, correct UI messages. Fixing it also surfaced a second latent bug the type change made visible: the missing-plumbing-point case was actually unreachable due to a check-ordering issue, fixed in the same pass.

DEVELOPER

"will the app only show product name as it is hard to understand which is toilet or vanity"

Outcome: added a category label above each line item in the bundle summary, so a product name like "Cimarron" is no longer ambiguous about which fixture slot it fills.

DEVELOPER

"2 toilet points... i can only control one"

Outcome: root-caused to the manual "Add plumbing point" button always defaulting to category "toilet" with no duplicate check, unlike the click-to-place path which already guarded against it. A second toilet point became inert dead data the engine silently ignored. Fixed by having the Add button default to the next missing category and disabling it once all three exist.

DEVELOPER

"clearance is making hard to place the plumbing point"

Outcome: reproduced live before proposing anything: repositioning an existing plumbing point updated its {x, y} but never recomputed which wall it faced, so the clearance validator silently checked the new position against the wrong wall. One-line fix reusing the already-tested wall-inference function, verified against exact click coordinates measured live in the browser.

DEVELOPER

"plumbing-62e0ed55-21ee-4052-af5b-ea56c9fd9d80 is it supposed to show"

Outcome: found the raw internal UUID spliced directly into four human-facing warning message templates in the fit validator. Removed the ID from all four (category alone already identifies the fixture), added four regression tests asserting the id string can never appear in a message again.

DEVELOPER

"can you also make the legend and product part more clean they are currently overlapping"

Outcome: measured the exact overlap (9.6px) with real DOM measurements before touching any CSS. Root cause was absolute positioning that sized the plan/legend stack to the wrong child's height. Fixed with CSS grid stacking, which sizes the shared area to the taller child at any width; re-measured afterward to confirm a clean, exact gap.

3.12 Deployment

DEVELOPER

"Fix the build bug and set up Vercel"

Outcome: found and fixed a real, platform-independent build failure first (a stray Vite/Vitest config import breaking type-checking), verified three ways (build, test suite, and the actual built output served locally). Deployment itself split along a hard boundary: Claude cannot complete GitHub/Vercel OAuth on the developer's behalf, so the developer created the repo and connected Vercel through the dashboard, while Claude pushed all code and verified the live deployment end to end once it was up.

4. Design Decision: No Live LLM Call at Runtime

The shipped app's chat layer and rationale text are deliberately deterministic — keyword and pattern matching (intent parsing) and templated logic (rationale generation), not a live call to an LLM. This was discussed explicitly and is a considered trade-off, not an oversight, reasoned through with Claude as part of the same spec-first process used for every feature.

Why deterministic, not a live API call

Reliability on demo day: a keyword/regex parser cannot time out, rate-limit, or hallucinate an unsupported action during a live grading session — every response is reproducible and instant.

A real infrastructure constraint, not a preference: this is a static, backend-less app in a public repository. There is nowhere to hold an API key that a live LLM call would need without exposing it client-side to anyone who opens the repo — doing this “properly” would require either an evaluator-supplied key or new backend/proxy infrastructure the project was never scoped to include.

Where AI actually did the work: AI (Claude) was the co-engineer for the entire build process itself — every schema, every solver rule, every UI component, every bug fix documented in Section 3 — rather than being invoked at runtime inside the shipped product. The application of AI here is in how the product was built, not only in what it calls while running.

This question was revisited more than once during the build and deliberately parked rather than resolved by default — a conscious scope decision, stated plainly here rather than left implicit.

5. Business & Sustainability: Water-Efficiency Scoring

Sustainability was built into the solver's scoring model from early in the build (*bundle-scorer.ts*), not added as a late cosmetic touch. Every candidate bundle's toilet, faucet, and shower are scored against real fixture specs — gallons per flush, gallons per minute — normalized against reference efficiency bounds, and that sub-score feeds both the bundle's overall ranking and a dedicated *sustainabilityScore* field.

Surfaced to the user, not just computed internally

That score was computed correctly from the very first working solve, but wasn't visible in the UI until it was deliberately surfaced as a “Water efficiency: NN%” line in the bundle summary — turning an internal metric already driving fixture selection into a decision factor the actual customer can see and weigh, directly supporting water-conscious purchasing within a KOHLER-fit bundle.

6. Summary

Bath Planner's build was driven by a consistent, explicit collaboration protocol between the developer and Claude: rank by dependency, spec before code, build modular pieces and integrate deliberately, verify live rather than trust tests alone, check blast radius before touching finished work, and never guess at a root cause. That same protocol carried through 35 build-board tasks, seven real user-reported bugs found by actually using the shipped app, and a full deployment pipeline — and it directly shaped one explicit, load-bearing product decision: keeping the shipped app's intelligence deterministic while using AI as the engineer for everything else.